CSE 4303
Data Structures

Topic: Elementary Graph Algorithms

(Reference: Ch 22.1-22.3 CLRS)

Sabbir Ahmed
Assistant Prof | CSE | IUT
sabbirahmed@iut-dhaka.edu

Breadth-first search (BFS)

- BFS is one of the simplest algorithms for graph searching and the archetype for many important graph algorithms.

- Prim's minimum-spanning tree algorithm and Dijkstra's single-source shortest-paths algorithm use ideas similar to those in breadth-first search.

Breadth-first search

- Given $G = (V, E)$ and ***source*** vertex s**, BFS** explores the edges of G to “discover” every vertex reachable from s**.**

Breadth-first search

- Given $G = (V, E)$ and ***source*** vertex s**, BFS** explores the edges of G to "discover" every vertex reachable from s**.**

- Computes **distance** (smallest # of edges) from s to each reachable vertex.

Breadth-first search

- Given $G = (V, E)$ and ***source*** vertex s**, BFS** explores the edges of G to "discover" every vertex reachable from s**.**

- Computes **distance** (smallest # of edges) from s to each reachable vertex.

- Produces "**breadth-first tree**" with root s having all reachable vertices.

- Discovers all vertices at **distance $\boldsymbol{k}$** from s **before** discovering any vertex at **distance $\boldsymbol{k+1}$**.

- Discovers all vertices at **distance k** from s **before** discovering any vertex at **distance $k+1$**.

- To keep track of progress, BFS **colors** each vertex ***white, gray,* or *black***.

  - All vertices **start *white***.
  - Later **become *gray***.
  - **Ends *black***.

- Our implementation stores G using **adjacency lists**.

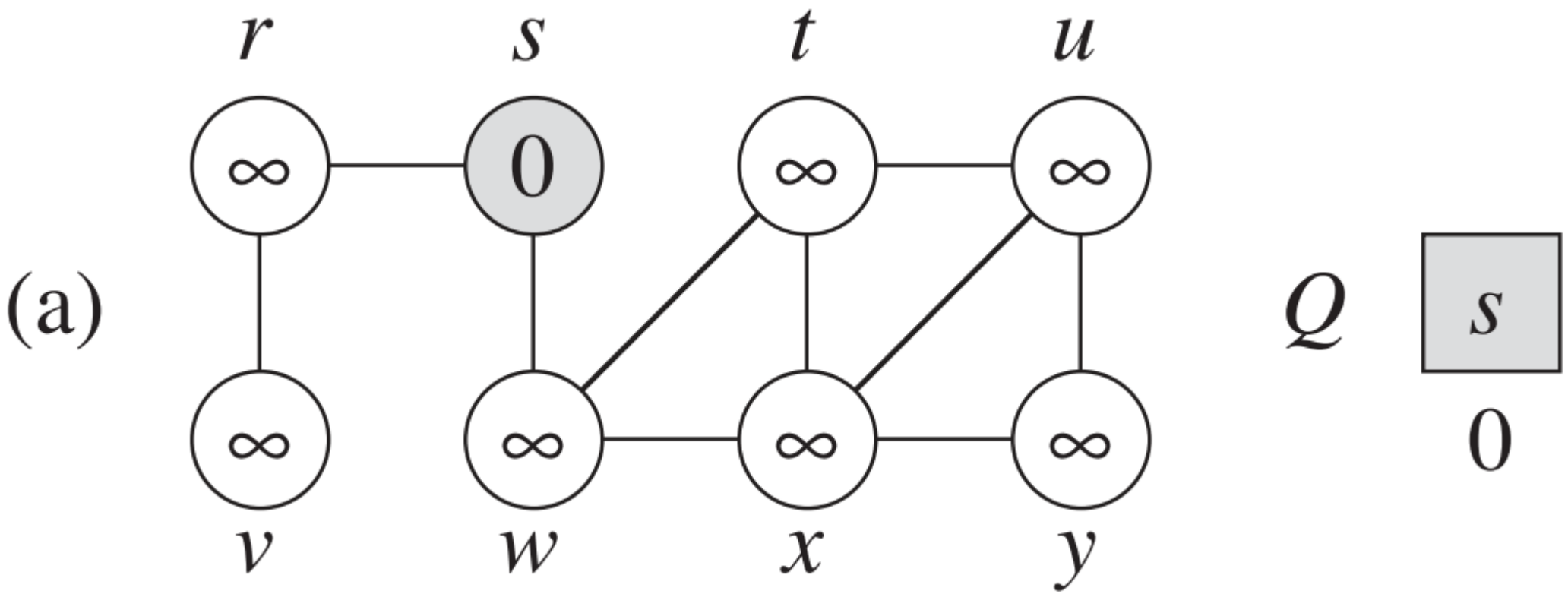
- Our implementation stores G using **adjacency lists**.

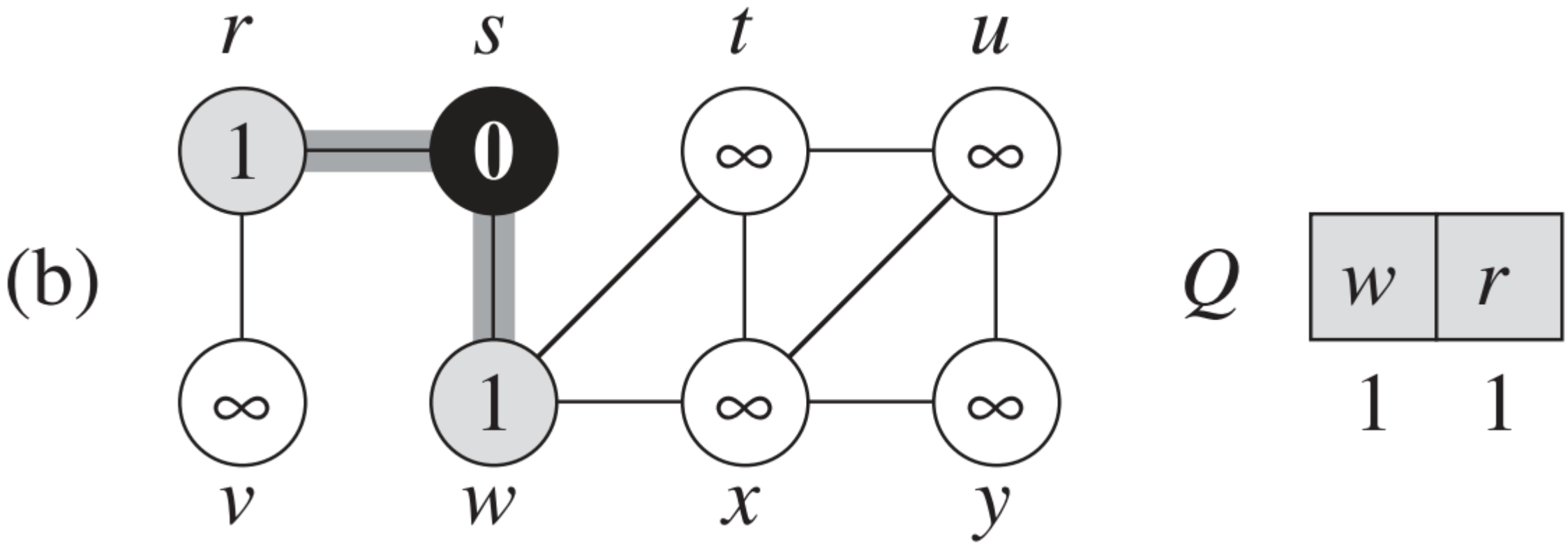
- Each vertex u has three attributes:
  - $u.color$: white/gray/black
  - $u.d$: distance from source s to u
  - $u.\pi$: predecessor of u

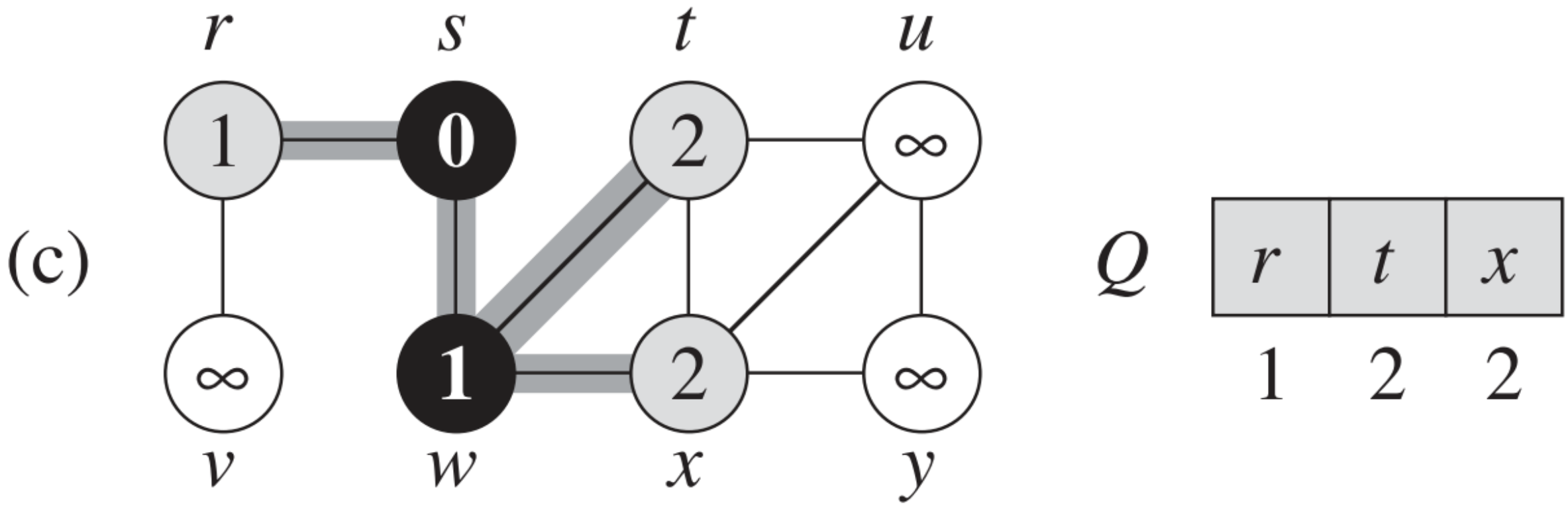
- Our implementation stores G using **adjacency lists**.

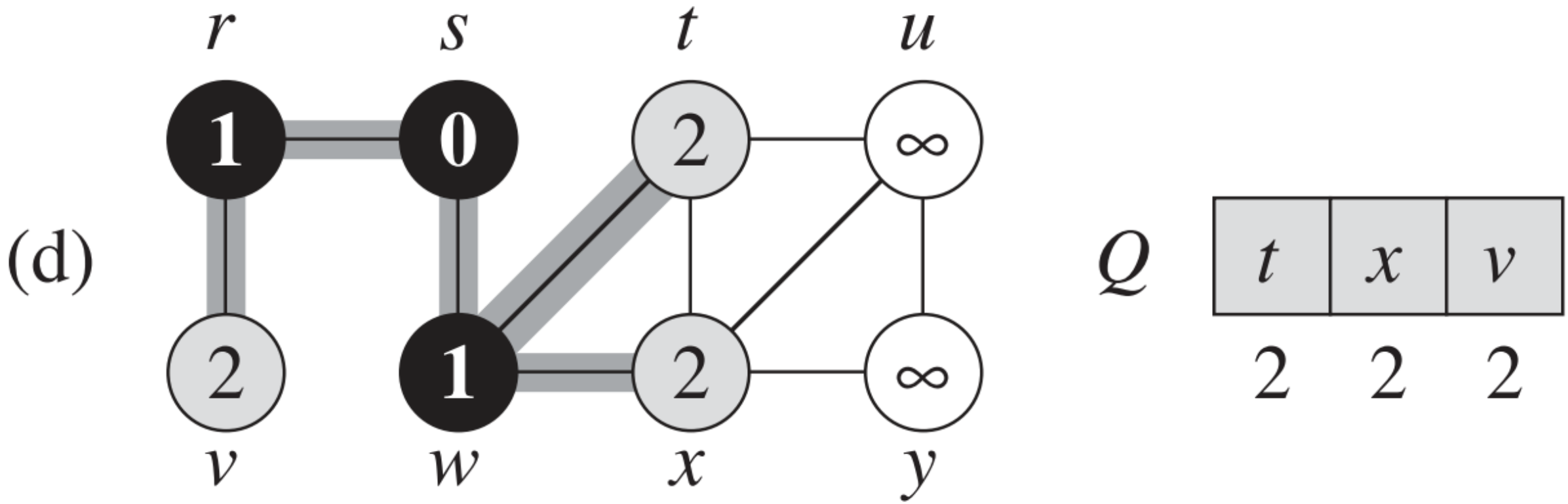
- Each vertex u has three attributes:
  - $u.color$: white/gray/black
  - $u.d$: distance from source s to u
  - $u.\pi$: predecessor of u

- BFS uses a **FIFO queue** Q to manage the ***gray*** nodes.

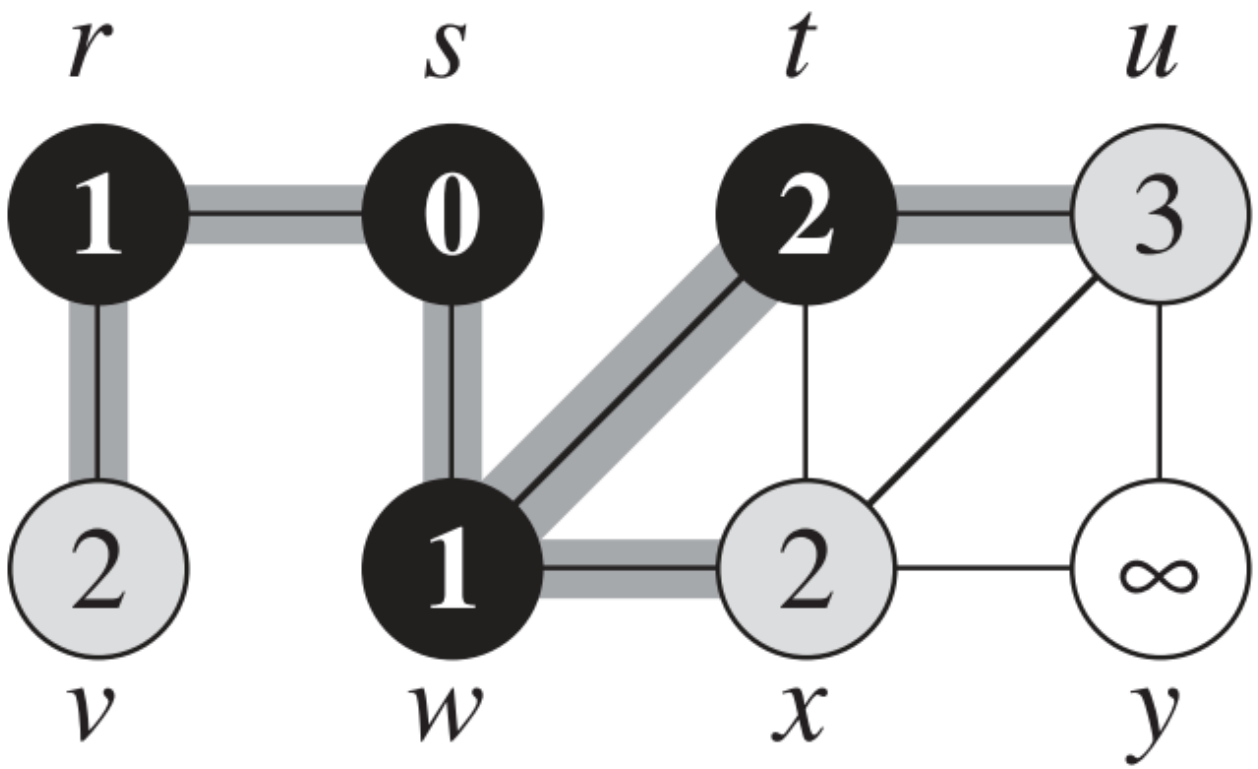

r
s
t
u
(a)
∞
0
∞
∞
∞
∞
∞
∞
v
w
x
y
Q
s
0


r
s
t
u
1
0
∞
∞
(b)
∞
1
∞
∞
v
w
x
y
Q
w
r
1
1


(c)
r
s
t
u
1
0
2
∞
∞
1
2
∞
v
w
x
y
Q
r
t
x
1
2
2

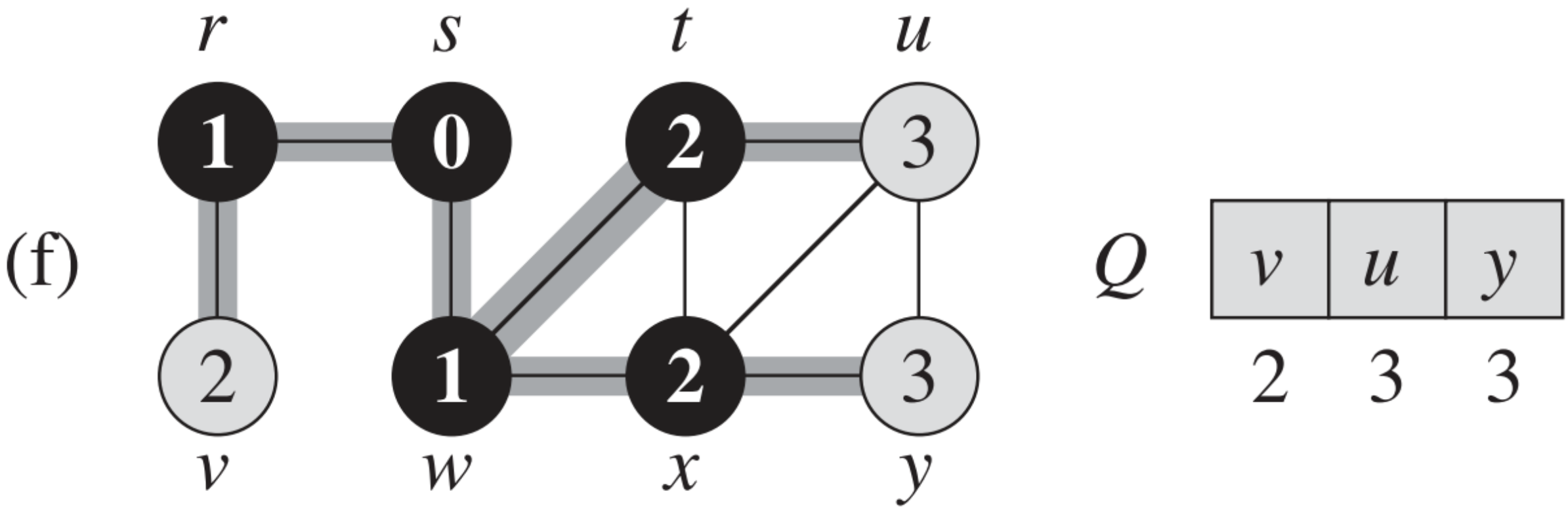

(d)
r
s
t
u
1
0
2
∞
2
1
2
∞
v
w
x
y
Q
t
x
v
2
2
2

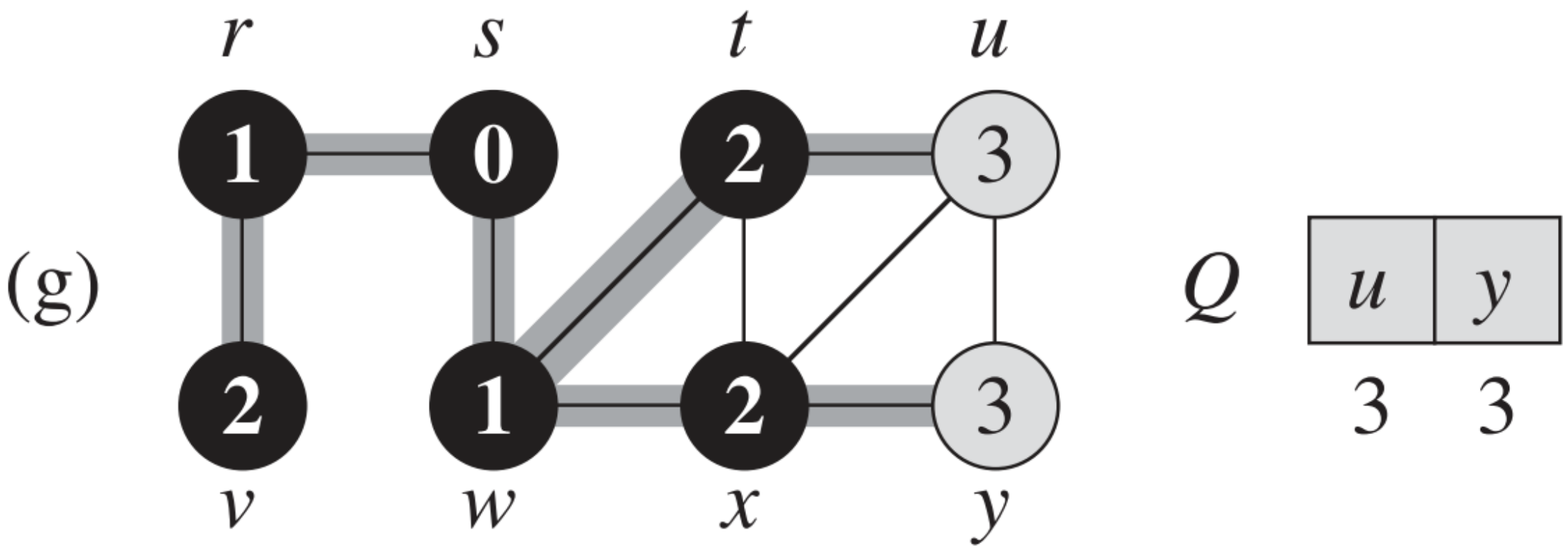
(e)

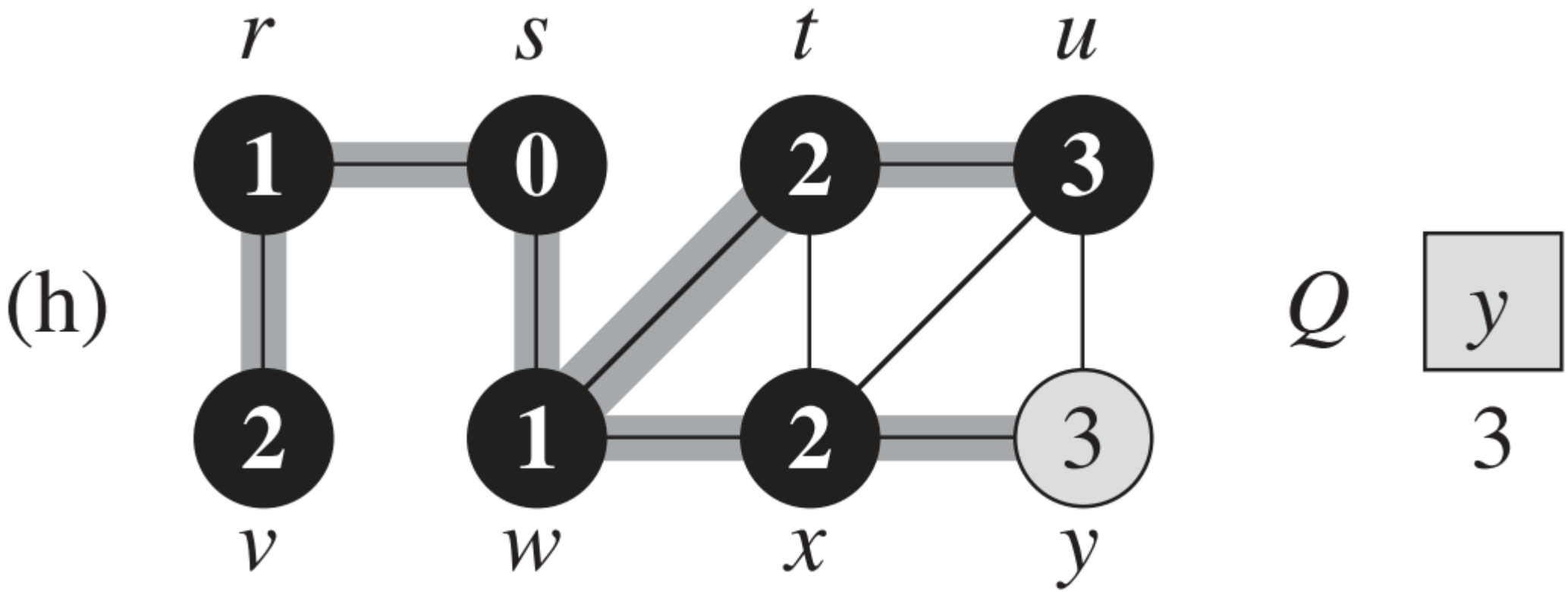


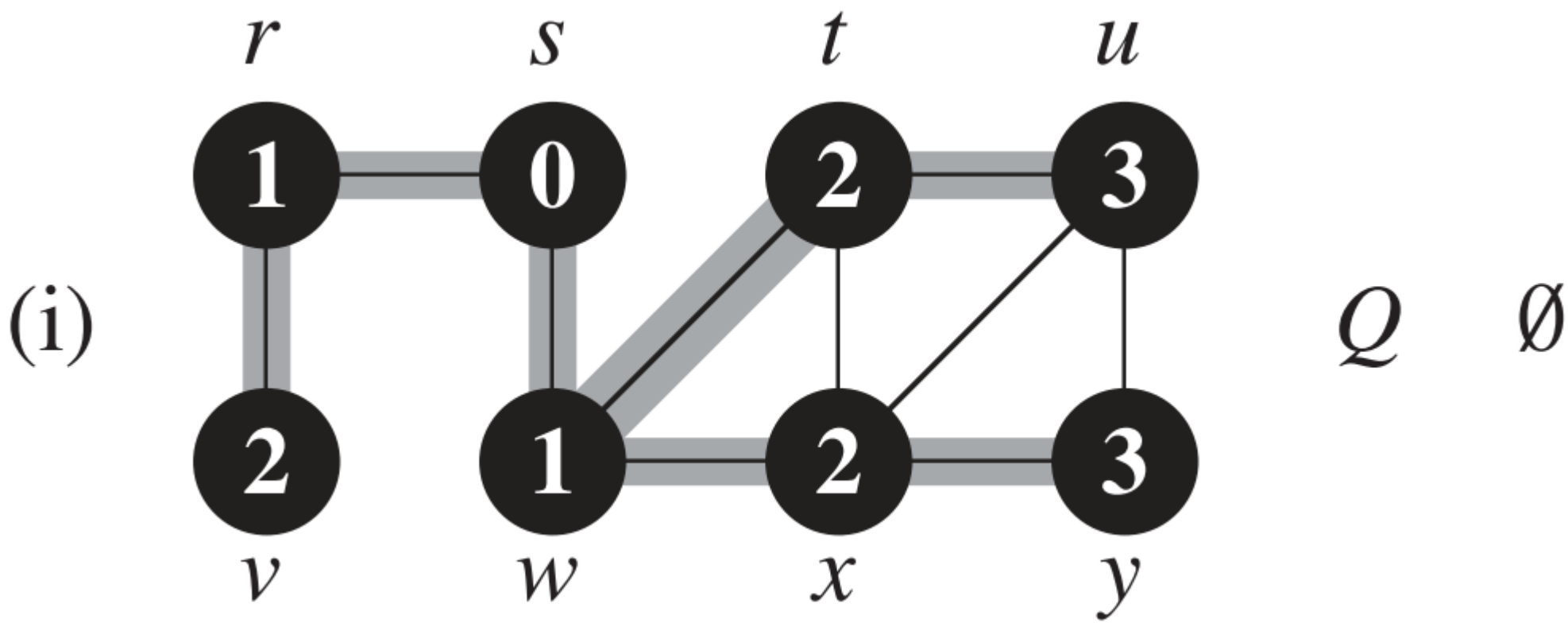
Q

x	v	u
2	2	3


r
s
t
u
1
0
2
3
(f)
2
1
2
3
v
w
x
y
Q
v
u
y
2
3
3


r
s
t
u
1
0
2
3
2
1
2
3
v
w
x
y
(g)
Q
u
y
3
3


r
s
t
u
1
0
2
3
(h)
2
1
2
3
v
w
x
y
Q
y
3


r
s
t
u
(i)
1
0
2
3
2
1
2
3
v
w
x
y
Q
∅

$\mathrm{BFS}(G, s)$

BFS(G, s)

1 **for** each vertex $u \in G.V - \{s\}$

```
BFS(G, s)
1   for each vertex u ∈ G.V − {s}
2       u.color = WHITE
3       u.d = ∞
4       u.π = NIL
```

```
BFS(G, s)
1  for each vertex u ∈ G.V − {s}
2       u.color = WHITE
3       u.d = ∞
4       u.π = NIL
5  s.color = GRAY
6  s.d = 0
7  s.π = NIL
```

```
BFS(G, s)
 1  for each vertex u ∈ G.V − {s}
 2      u.color = WHITE
 3      u.d = ∞
 4      u.π = NIL
 5  s.color = GRAY
 6  s.d = 0
 7  s.π = NIL
 8  Q = ∅
 9  ENQUEUE(Q, s)
```

```
BFS(G, s)
 1  for each vertex u ∈ G.V − {s}
 2       u.color = WHITE
 3       u.d = ∞
 4       u.π = NIL
 5  s.color = GRAY
 6  s.d = 0
 7  s.π = NIL
 8  Q = ∅
 9  ENQUEUE(Q, s)
10  while Q ≠ ∅
```

```
BFS(G, s)
 1   for each vertex u ∈ G.V − {s}
 2       u.color = WHITE
 3       u.d = ∞
 4       u.π = NIL
 5   s.color = GRAY
 6   s.d = 0
 7   s.π = NIL
 8   Q = ∅
 9   ENQUEUE(Q, s)
10   while Q ≠ ∅
11       u = DEQUEUE(Q)
```

```
BFS(G, s)
 1  for each vertex u ∈ G.V − {s}
 2      u.color = WHITE
 3      u.d = ∞
 4      u.π = NIL
 5  s.color = GRAY
 6  s.d = 0
 7  s.π = NIL
 8  Q = ∅
 9  ENQUEUE(Q, s)
10  while Q ≠ ∅
11      u = DEQUEUE(Q)
12      for each v ∈ G.Adj[u]
```

```
BFS(G, s)
 1  for each vertex u ∈ G.V − {s}
 2      u.color = WHITE
 3      u.d = ∞
 4      u.π = NIL
 5  s.color = GRAY
 6  s.d = 0
 7  s.π = NIL
 8  Q = ∅
 9  ENQUEUE(Q, s)
10  while Q ≠ ∅
11      u = DEQUEUE(Q)
12      for each v ∈ G.Adj[u]
13          if v.color == WHITE
14              v.color = GRAY
15              v.d = u.d + 1
16              v.π = u
```

```
BFS(G, s)
 1  for each vertex u ∈ G.V − {s}
 2      u.color = WHITE
 3      u.d = ∞
 4      u.π = NIL
 5  s.color = GRAY
 6  s.d = 0
 7  s.π = NIL
 8  Q = ∅
 9  ENQUEUE(Q, s)
10  while Q ≠ ∅
11      u = DEQUEUE(Q)
12      for each v ∈ G.Adj[u]
13          if v.color == WHITE
14              v.color = GRAY
15              v.d = u.d + 1
16              v.π = u
17              ENQUEUE(Q, v)
```

```
BFS(G, s)
 1  for each vertex u ∈ G.V − {s}
 2      u.color = WHITE
 3      u.d = ∞
 4      u.π = NIL
 5  s.color = GRAY
 6  s.d = 0
 7  s.π = NIL
 8  Q = ∅
 9  ENQUEUE(Q, s)
10  while Q ≠ ∅
11      u = DEQUEUE(Q)
12      for each v ∈ G.Adj[u]
13          if v.color == WHITE
14              v.color = GRAY
15              v.d = u.d + 1
16              v.π = u
17              ENQUEUE(Q, v)
18      u.color = BLACK
```

- Running time: $O(V + E)$
  - Initialization : $O(V)$
  - For each v, each adjacency list is checked at most once.
  - Hence the cost of the **while** loop is $O(V + E)$.

Depth First Search

Depth-first search (DFS)

- Strategy: to search ***deeper*** in the graph whenever possible.

- DFS ***explores edges out of the most recently discovered vertex*** v that still has unexplored edges leaving it.

Depth-first search (DFS)

- Strategy: to search ***deeper*** in the graph whenever possible.

- DFS ***explores edges out of the most recently discovered vertex*** v that still has unexplored edges leaving it.

- Once all of v's edges have been explored, the search ***backtracks*** to explore edges leaving the vertex from which v was discovered

Depth-first search (DFS)

- As in BFS, DFS **colors** vertices during the search to indicate their state.

- Initially ***white***, is ***grayed*** when it is discovered in the search, and is ***blackened*** when it is finished

Depth-first search (DFS)

- DFS also **timestamps** each vertex. Each vertex v has two timestamps:

Depth-first search (DFS)

- DFS also **timestamps** each vertex. Each vertex v has two timestamps:
  - First timestamp $v.d$ records when v is first discovered (and grayed)

Depth-first search (DFS)

- DFS also ***timestamps*** each vertex. Each vertex v has two timestamps:
  - First timestamp $\boldsymbol{v.d}$ records when v is first discovered (and grayed)
  - Second timestamp $\boldsymbol{v.f}$ records when the search finishes examining v's adjacency list (and blackens v).

Depth-first search (DFS)

- DFS also ***timestamps*** each vertex. Each vertex v has two timestamps:
  - First timestamp $\boldsymbol{v.d}$ records when v is first discovered (and grayed)
  - Second timestamp $\boldsymbol{v.f}$ records when the search finishes examining v's adjacency list (and blackens v).

- These timestamps provide important information about the structure of the graph.

```
DFS(G)
1   for each vertex u ∈ G.V
2       u.color = WHITE
3       u.π = NIL
```

```
DFS(G)
1   for each vertex u ∈ G.V
2       u.color = WHITE
3       u.π = NIL
4   time = 0
5   for each vertex u ∈ G.V
6       if u.color == WHITE
7           DFS-VISIT(G, u)
```

```
DFS(G)
1  for each vertex u ∈ G.V
2      u.color = WHITE
3      u.π = NIL
4  time = 0
5  for each vertex u ∈ G.V
6      if u.color == WHITE
7          DFS-VISIT(G, u)
```

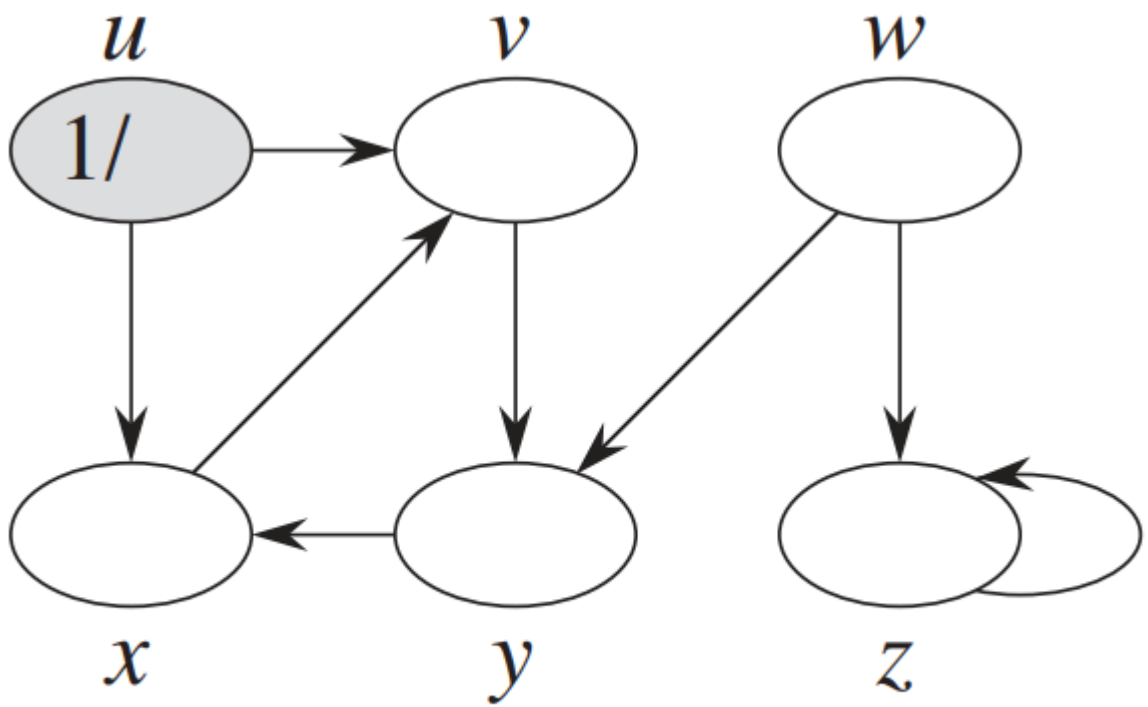
```
DFS-VISIT(G, u)
1  time = time + 1          // white vertex u has just been discovered
2  u.d = time
3  u.color = GRAY
```

```
DFS(G)
1  for each vertex u ∈ G.V
2      u.color = WHITE
3      u.π = NIL
4  time = 0
5  for each vertex u ∈ G.V
6      if u.color == WHITE
7          DFS-VISIT(G, u)

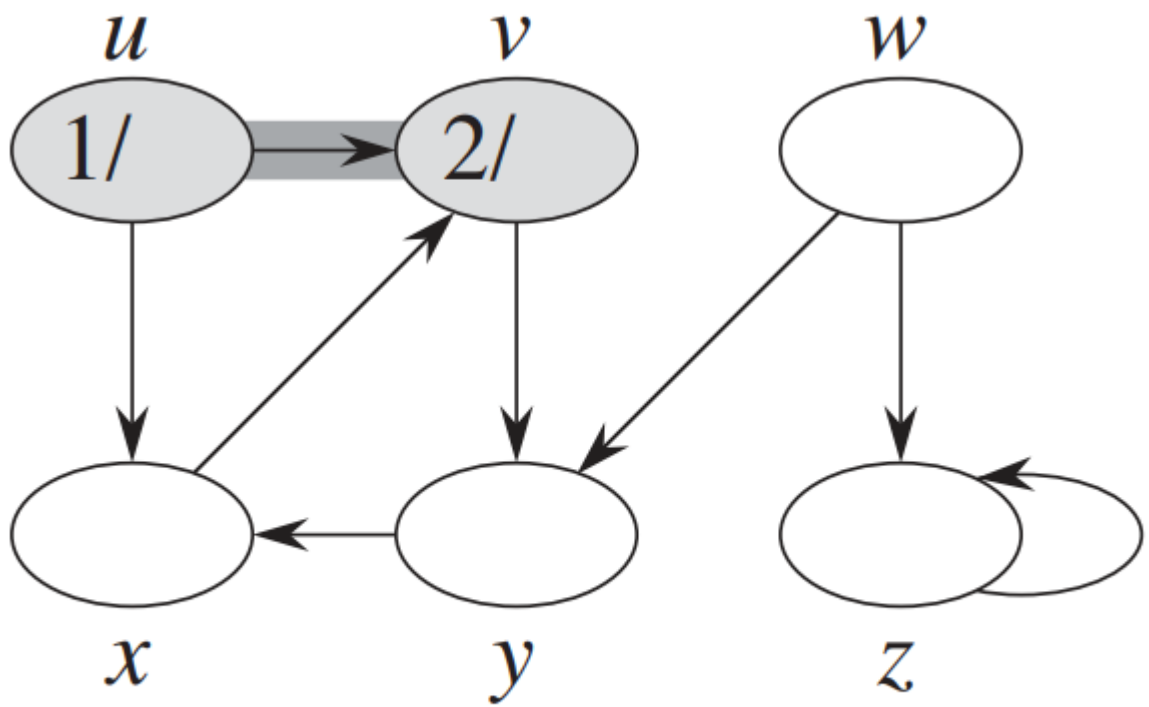
DFS-VISIT(G, u)
 1  time = time + 1                  // white vertex u has just been discovered
 2  u.d = time
 3  u.color = GRAY
 4  for each v ∈ G.Adj[u]            // explore edge (u, v)
 5      if v.color == WHITE
 6          v.π = u
 7          DFS-VISIT(G, v)
```

```
DFS(G)
1  for each vertex u ∈ G.V
2      u.color = WHITE
3      u.π = NIL
4  time = 0
5  for each vertex u ∈ G.V
6      if u.color == WHITE
7          DFS-VISIT(G, u)

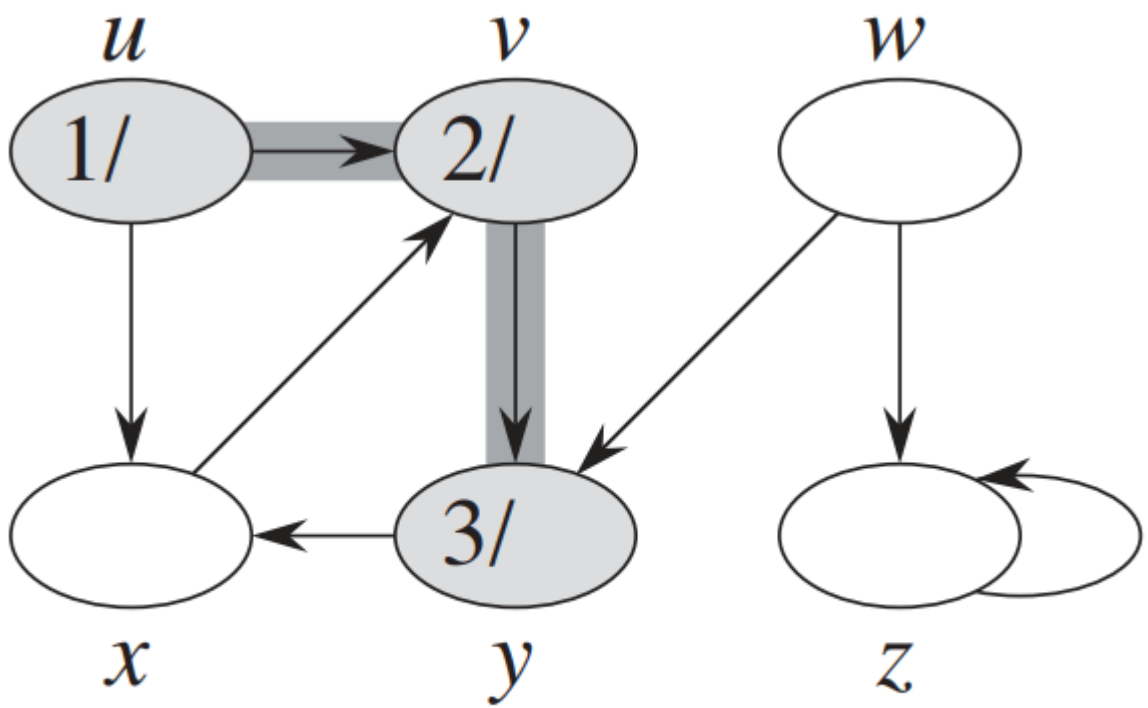
DFS-VISIT(G, u)
 1  time = time + 1                 // white vertex u has just been discovered
 2  u.d = time
 3  u.color = GRAY
 4  for each v ∈ G.Adj[u]           // explore edge (u, v)
 5      if v.color == WHITE
 6          v.π = u
 7          DFS-VISIT(G, v)
 8  u.color = BLACK                 // blacken u; it is finished
 9  time = time + 1
10  u.f = time
```



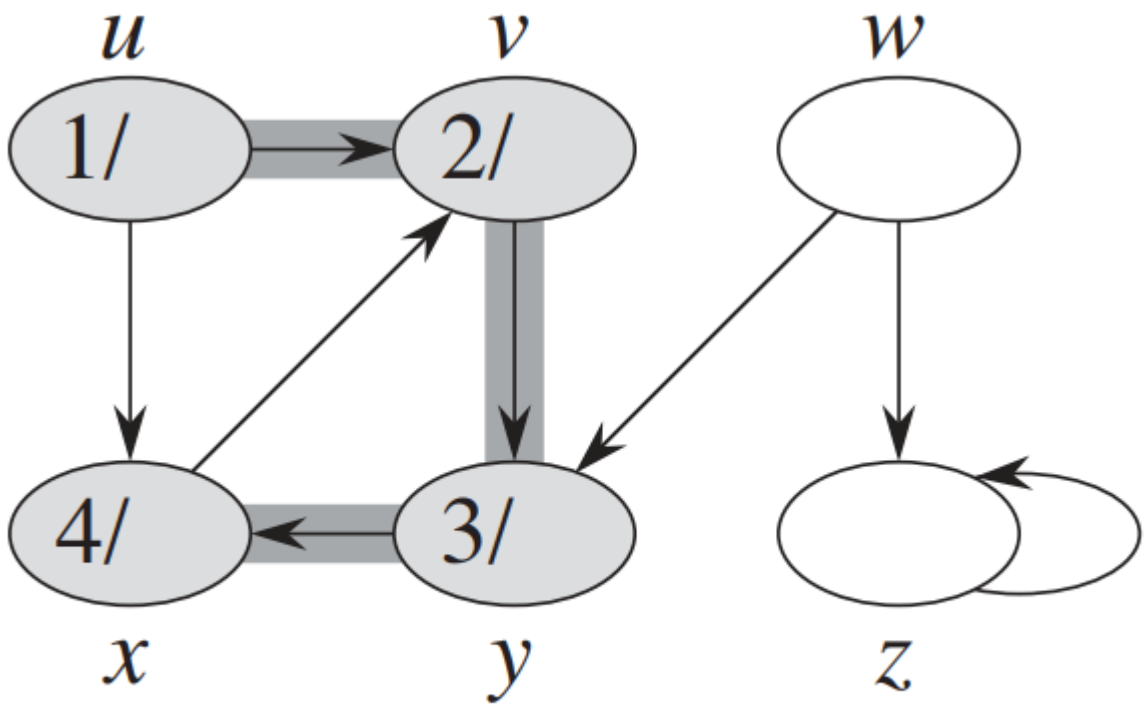
(a)



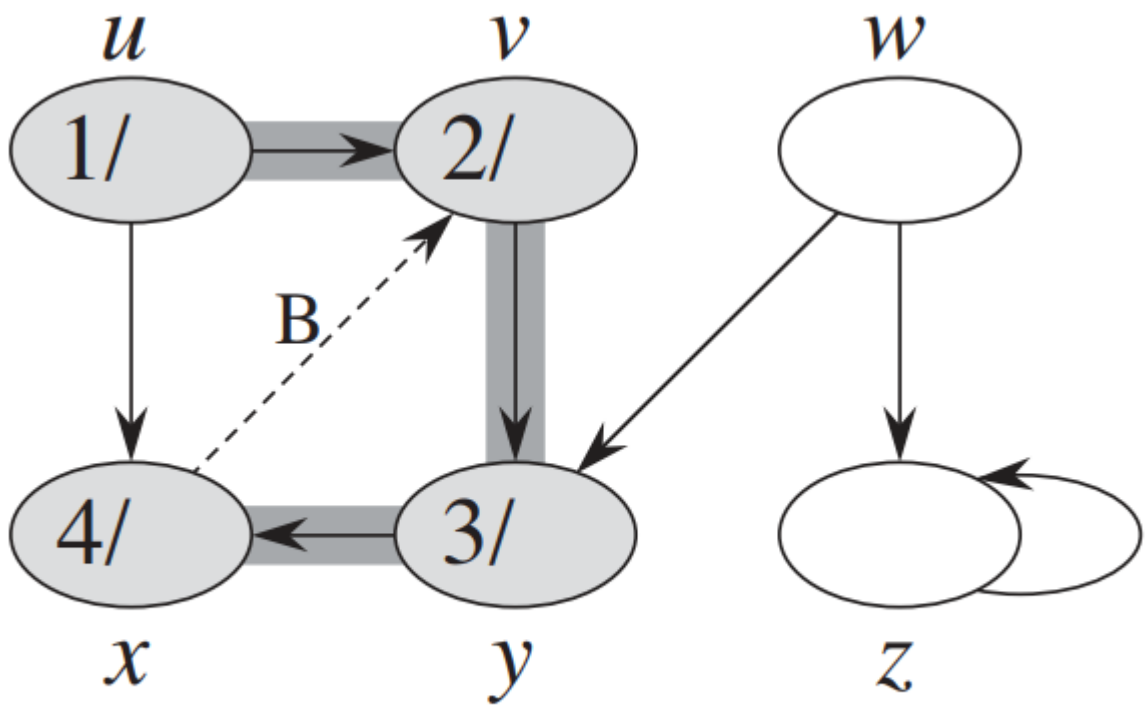
(b)



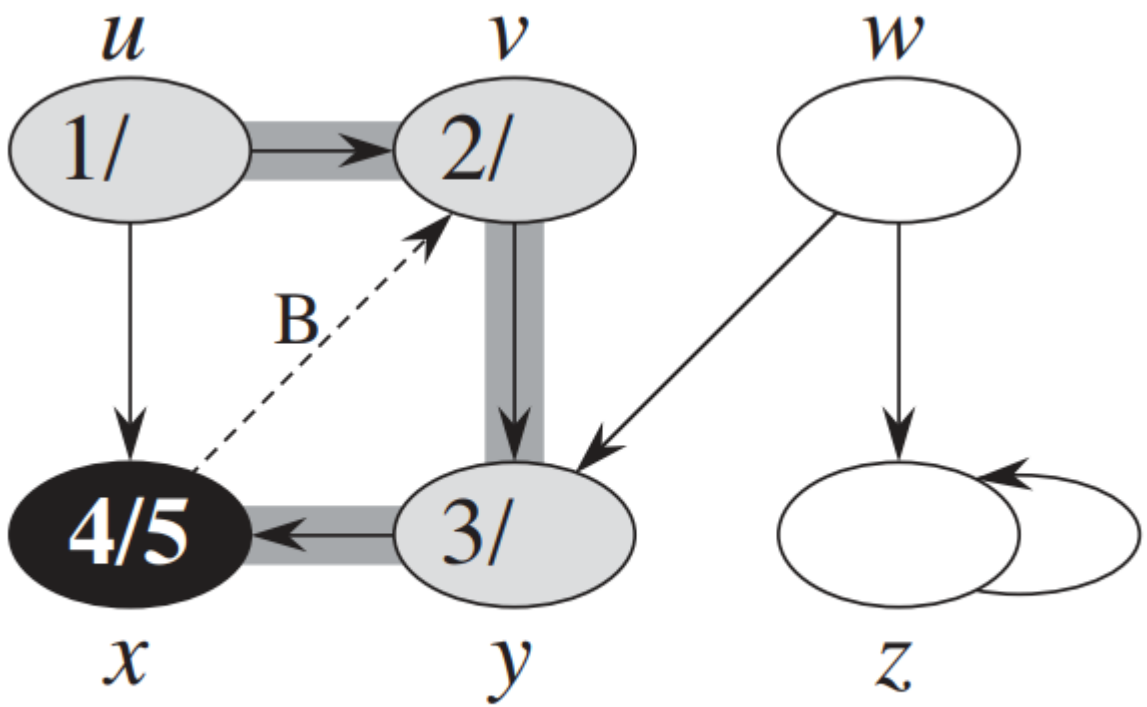
(c)



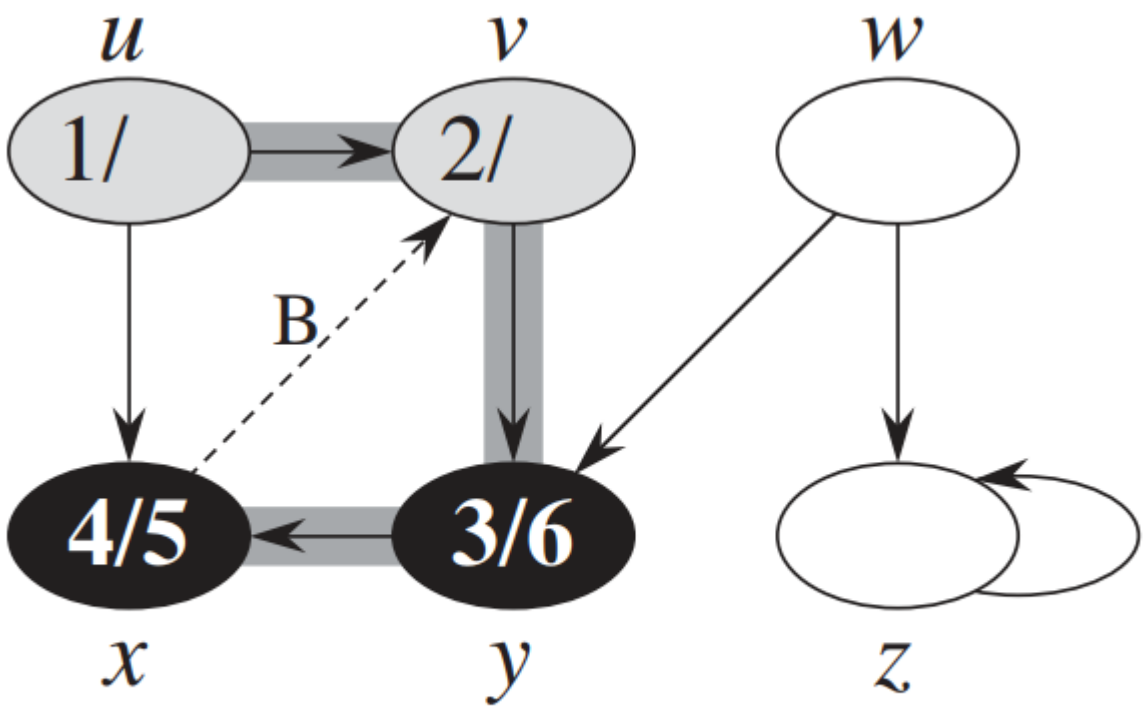
(d)



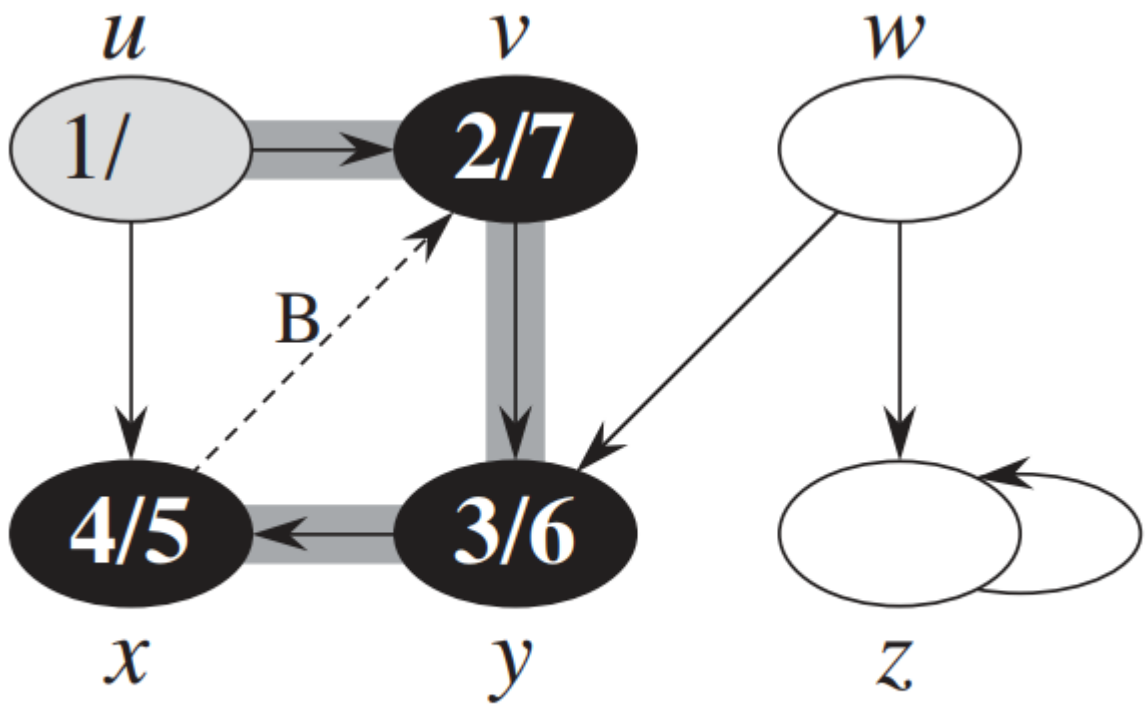
(e)



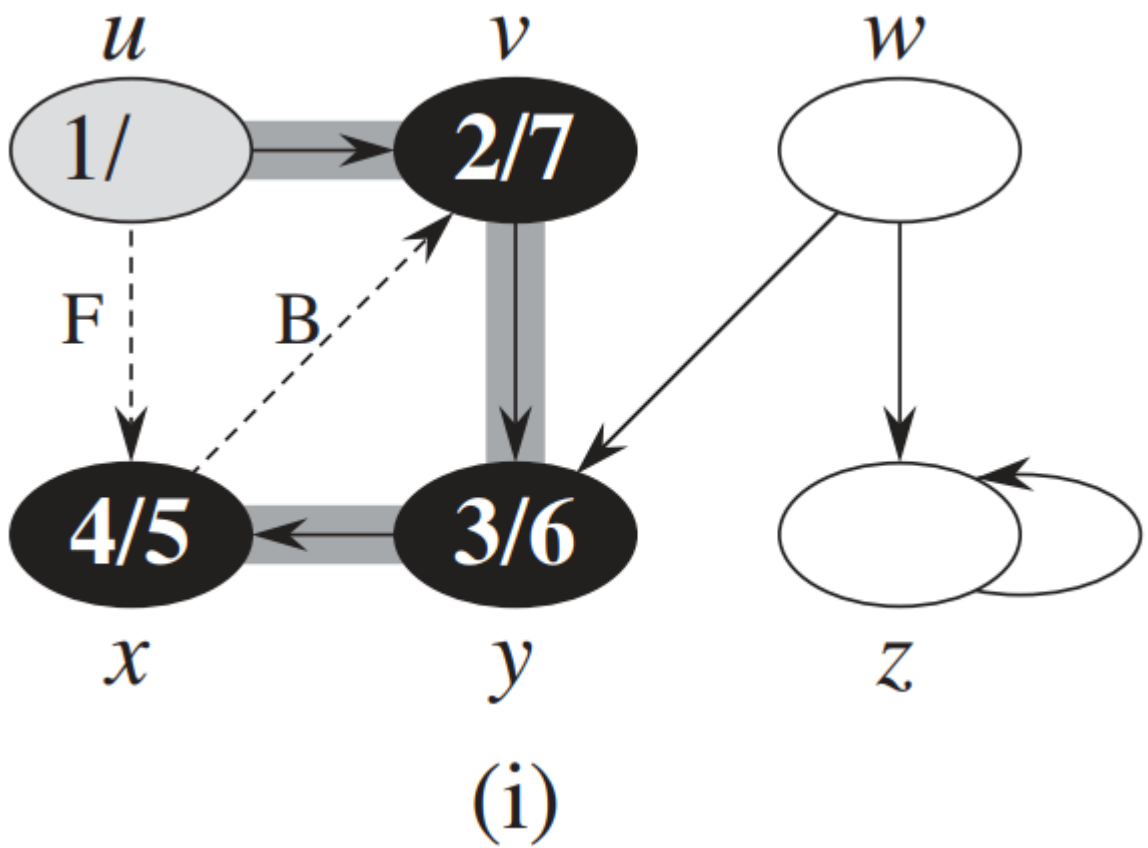
(f)

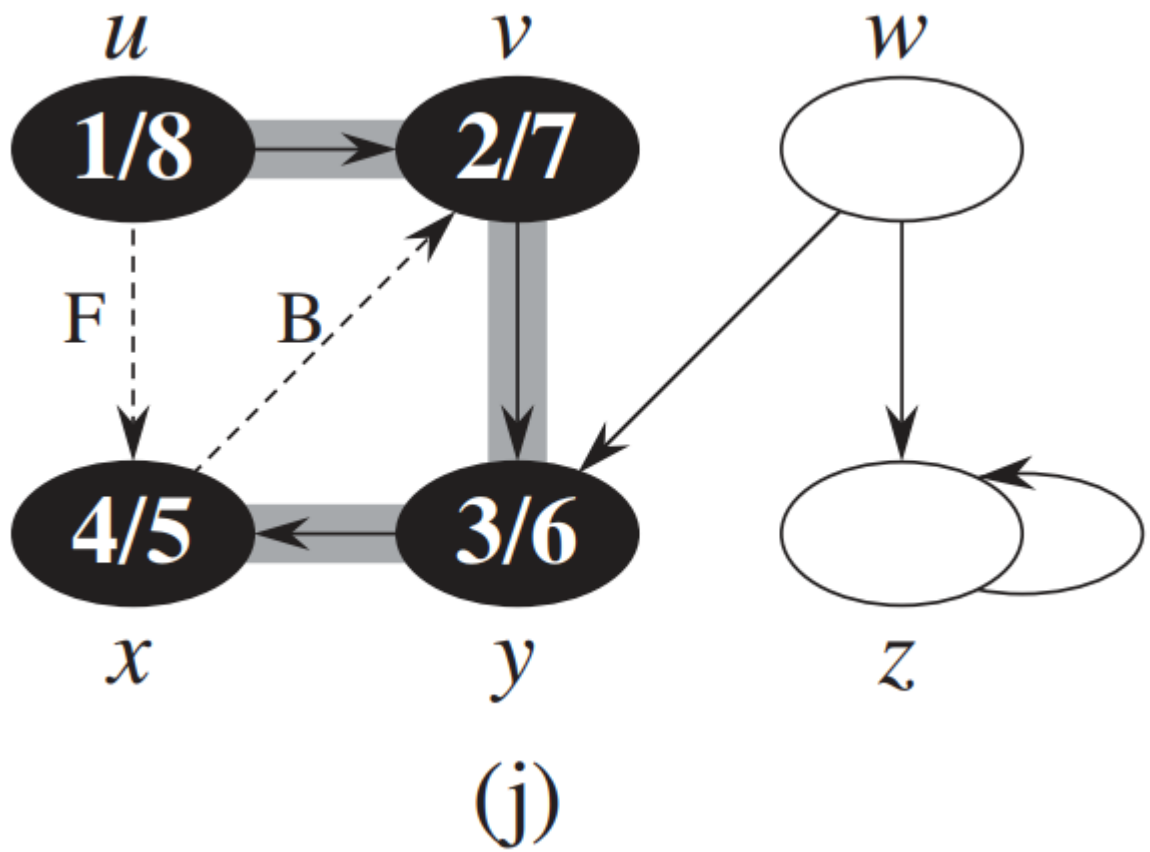


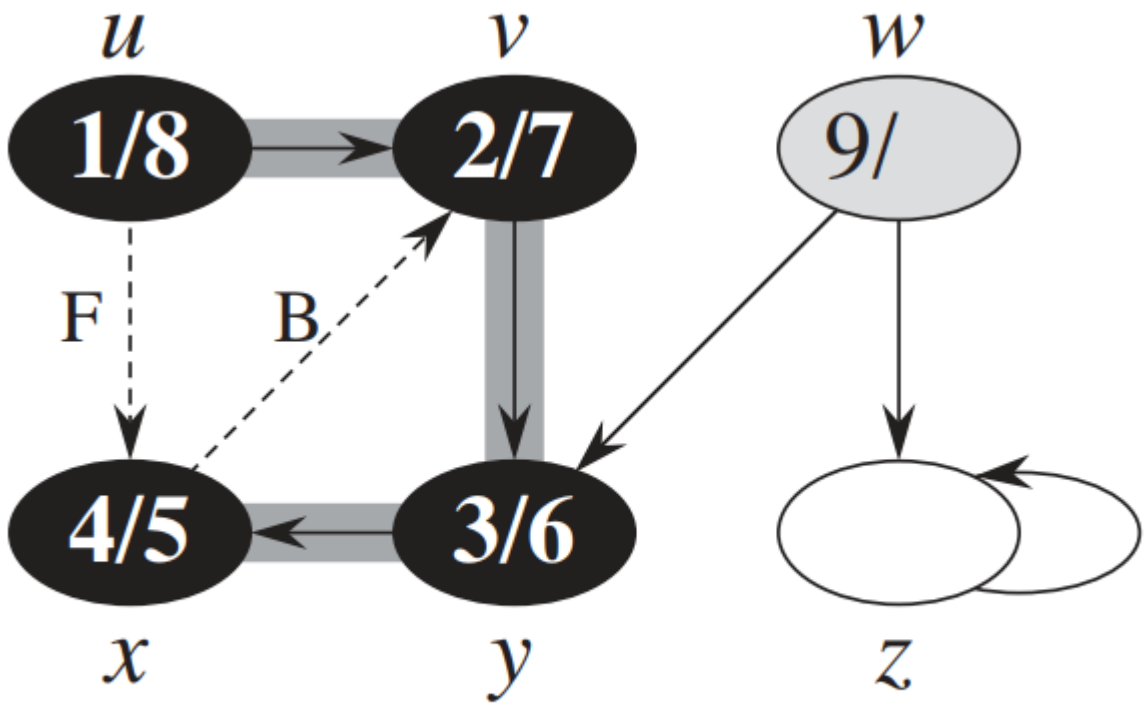
(g)



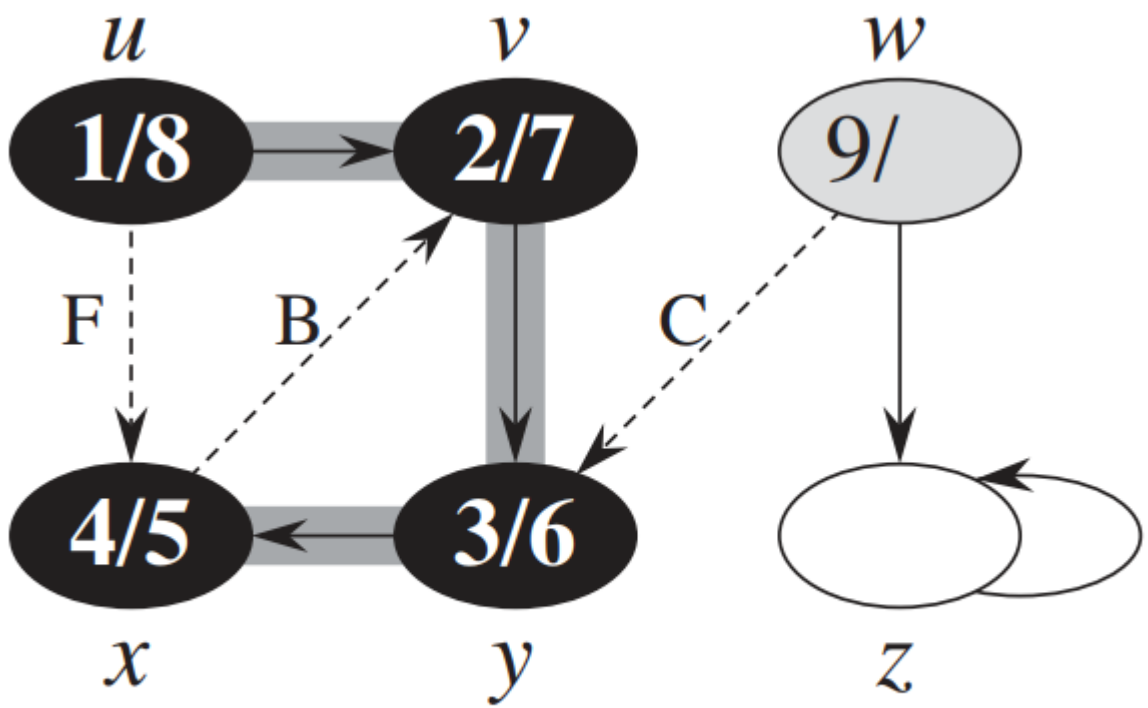
(h)


u
v
w
1/
2/7
F
B
4/5
3/6
x
y
z
(i)

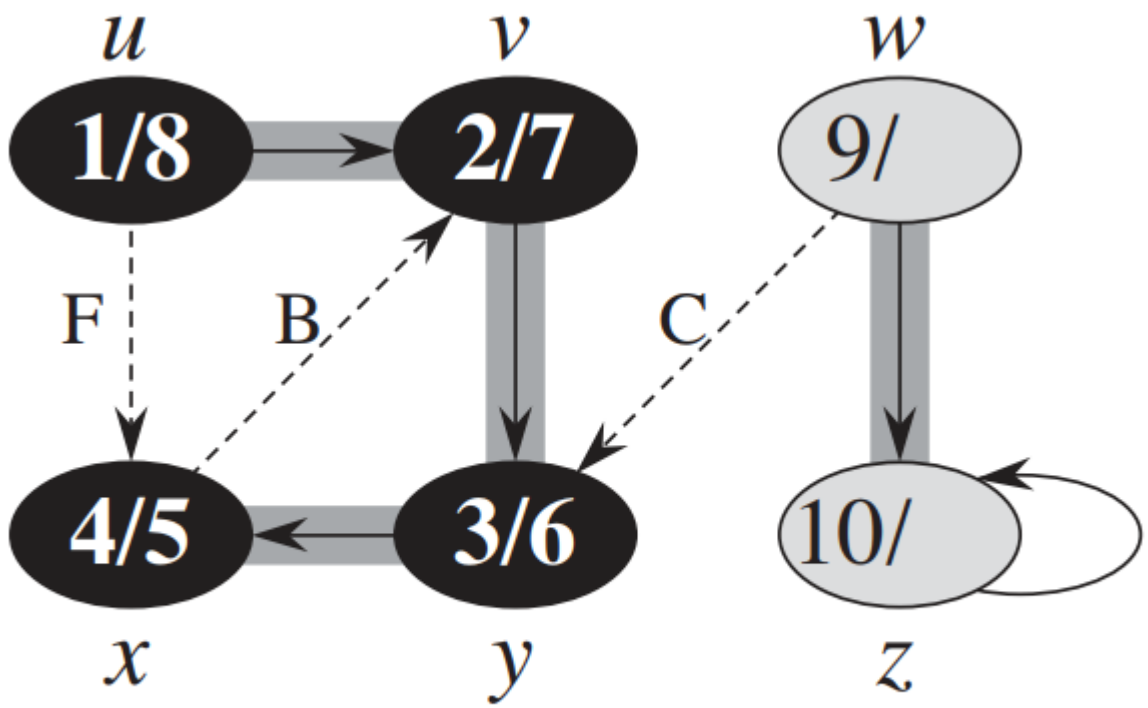

u
v
w
1/8
2/7
F
B
4/5
3/6
x
y
z
(j)



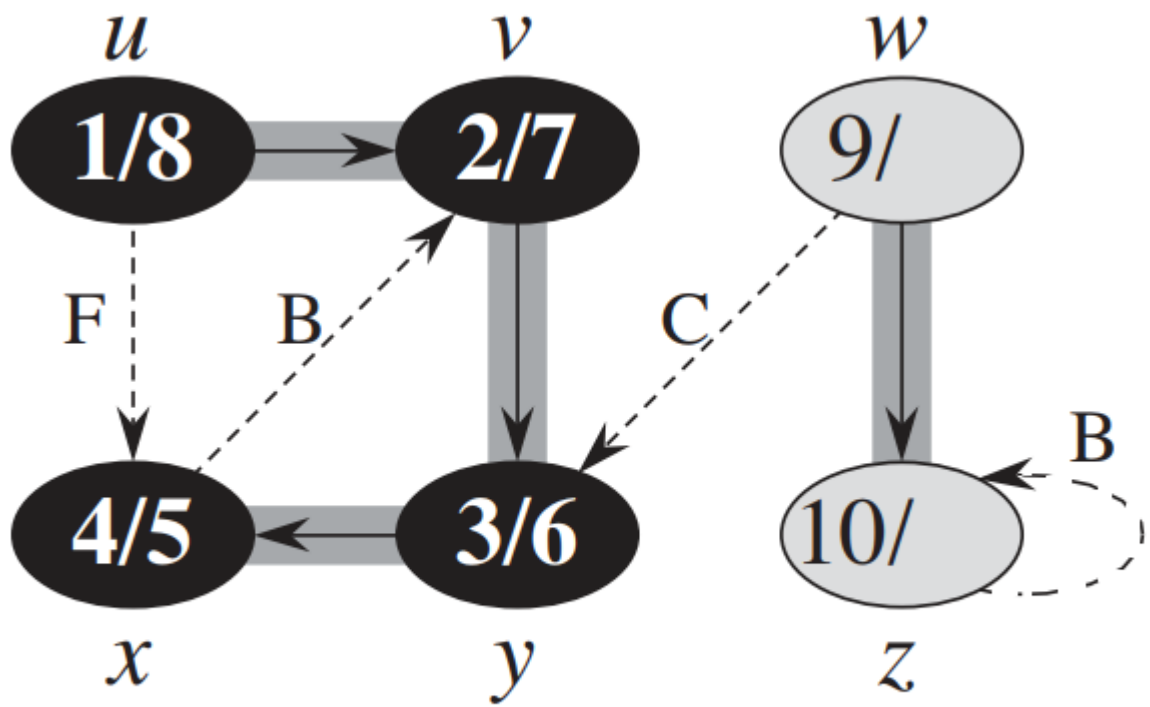
(k)



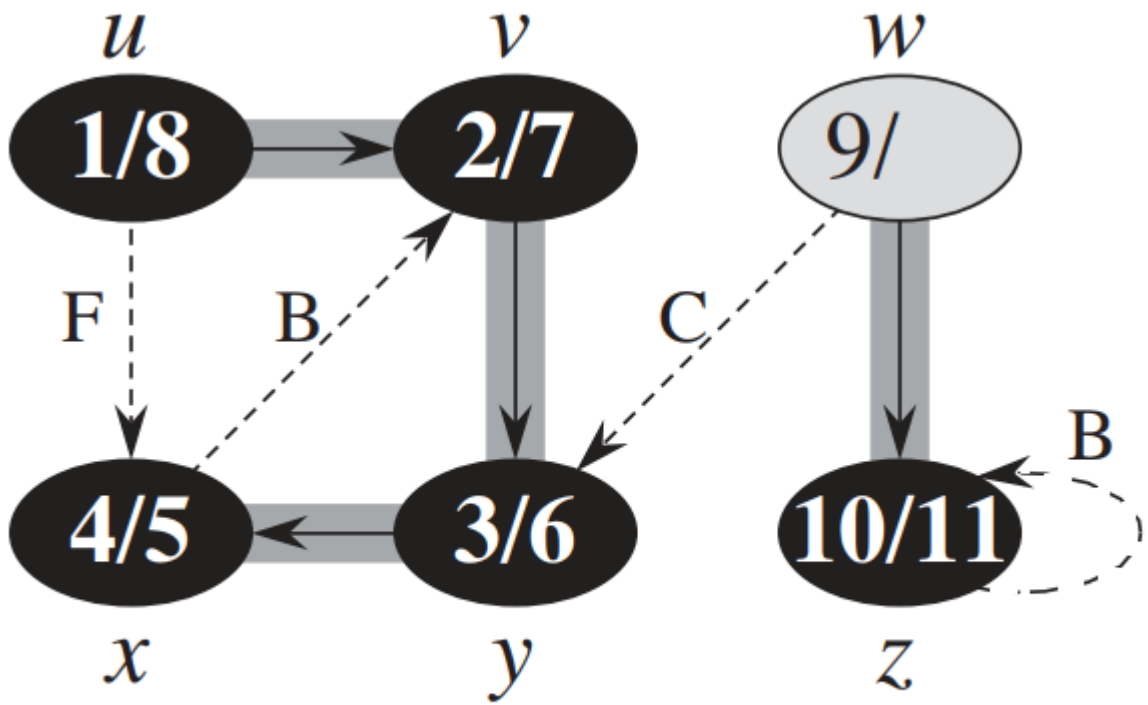
(l)



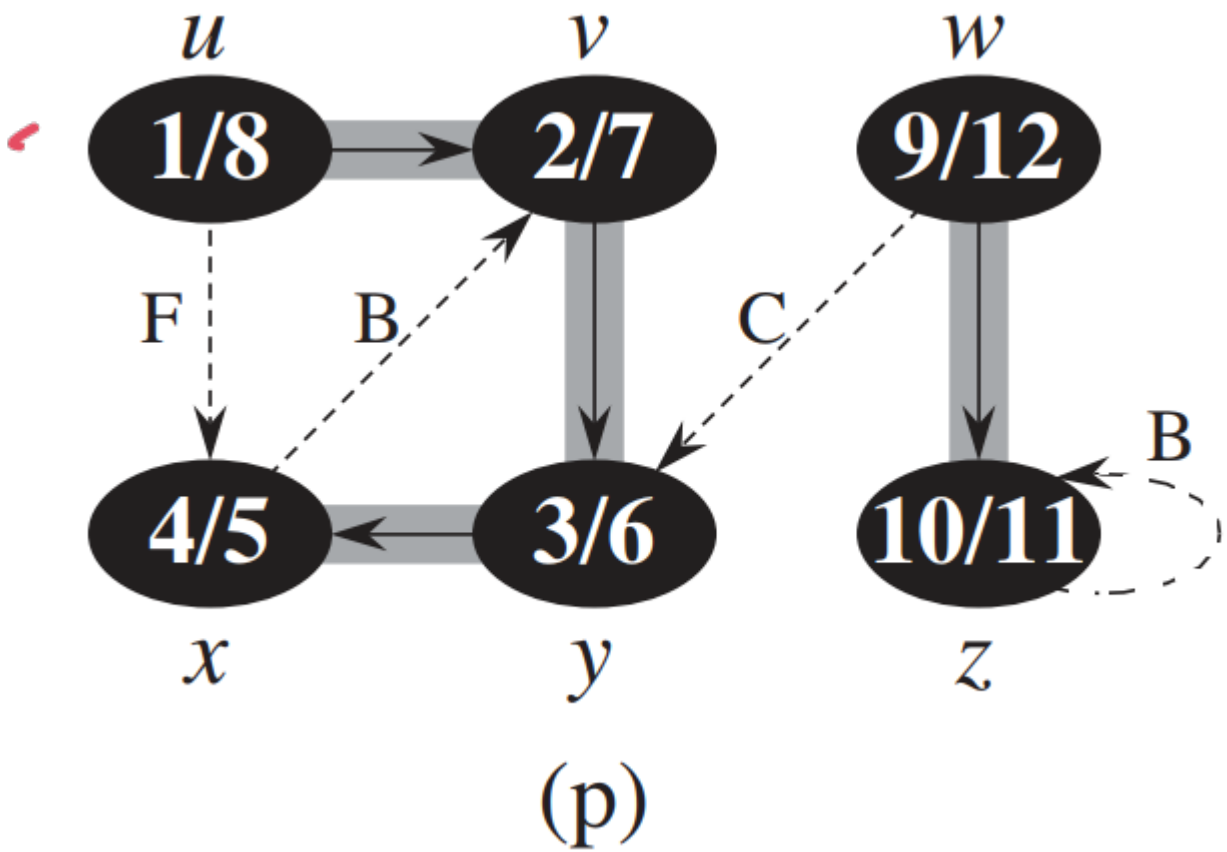
(m)



(n)



(o)



(p)

- DFS can be used to **classify the edges** of the input graph.
- The type of each edge can provide important information about a graph.
- For example, a directed graph is acyclic if and only if a depth-first search yields no ‘back’ edges.

- Four edge types in terms of DFS:

1. **Tree edges** are edges in the depth-first forest. Edge (u, v) is a tree edge if v was ***first discovered*** by exploring edge (u, v).

- Four edge types in terms of DFS:

1. **Tree edges** are edges in the depth-first forest. Edge (u, v) is a tree edge if v was ***first discovered*** by exploring edge (u, v).

2. **Back edges** are those edges (u, v) ***connecting a vertex u to an ancestor v*** in a depth-first tree.

   (We consider self-loops, which may occur in directed graphs, to be back edges.)

- Four edge types in terms of DFS:

  3. Forward edges are those nontree edges (u, v) connecting a vertex u to a ***descendant*** v in a depth-first tree.

- Four edge types in terms of DFS:

  3. Forward edges are those nontree edges (u, v) connecting a vertex u to a ***descendant*** v in a depth-first tree.

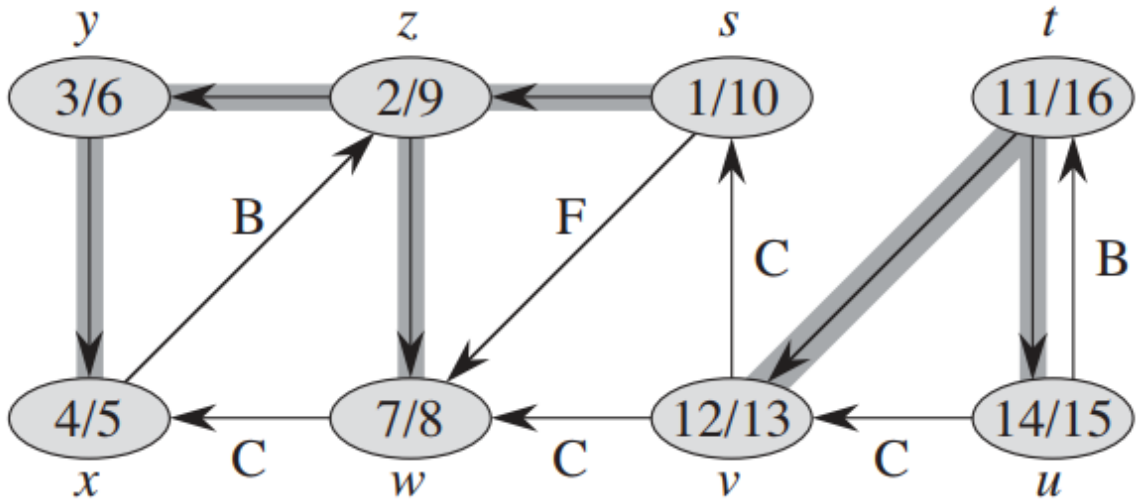
  4. Cross edges are all other edges. They can go between vertices in the same depth-first tree, as long as one vertex is not an ancestor of the other, or they can go between vertices in different depth-first trees.

- The DFS algorithm has enough information to classify some edges as it encounters them.

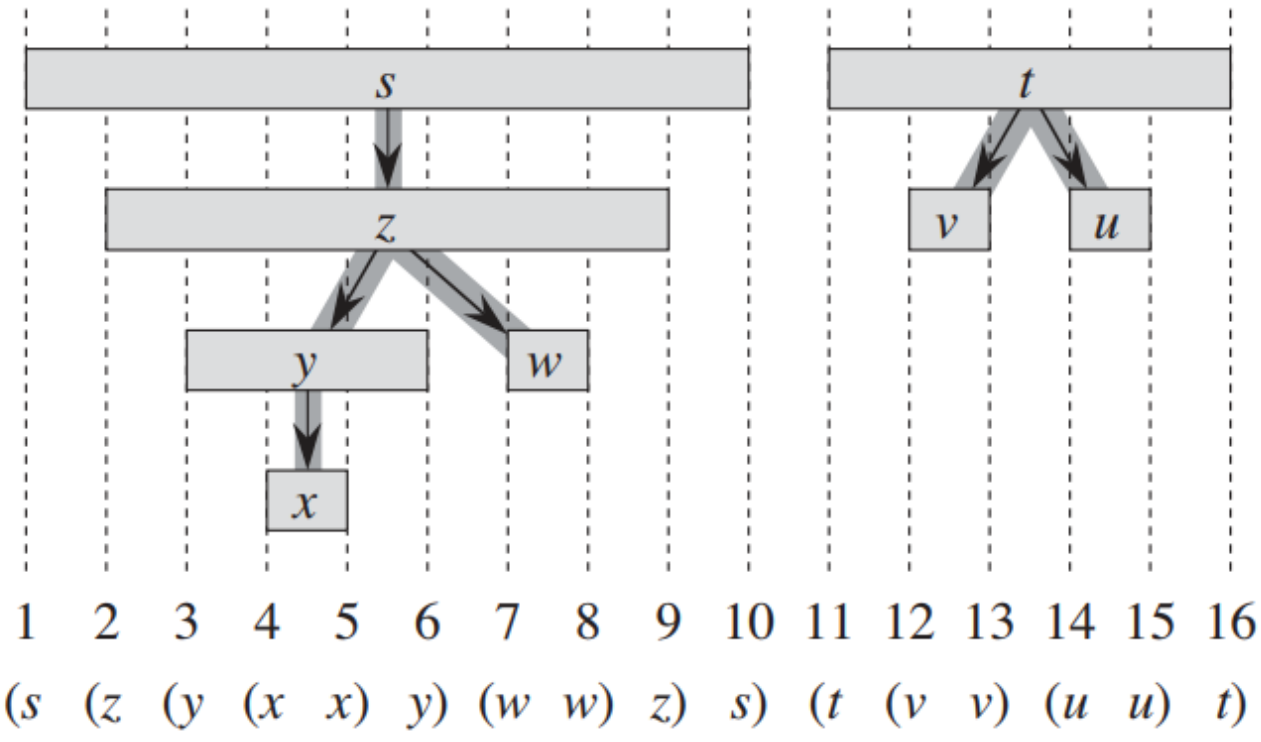
- The key idea is that when we first explore an edge (u, v), the color of vertex v tells us something about the edge:

  1. WHITE indicates a *tree edge*,
  2. GRAY indicates a *back edge*, and
  3. BLACK indicates a *forward edge* or *cross edge*.

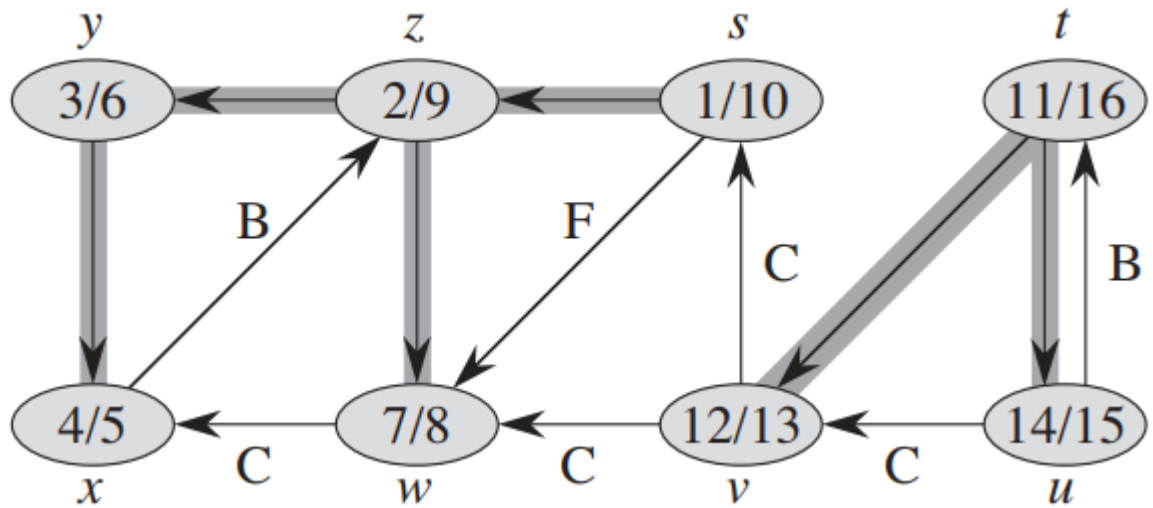
(a)


y
z
s
t
3/6
2/9
1/10
11/16
B
F
C
B
4/5
7/8
12/13
14/15
C
C
C
x
w
v
u

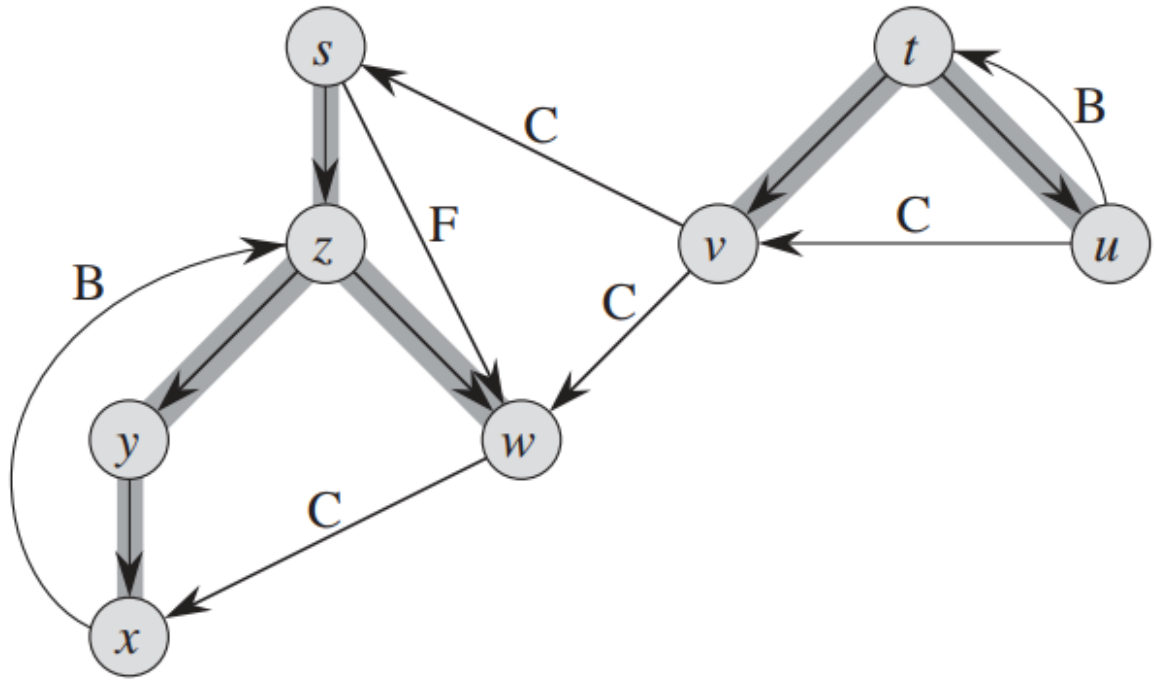
(b)


s
t
z
v
u
y
w
x
1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16
(s (z (y (x x) y) (w w) z) s) (t (v v) (u u) t)

(a)


y
z
s
t
3/6
2/9
1/10
11/16
B
F
C
B
4/5
7/8
12/13
14/15
C
C
C
x
w
v
u

(c)


s
t
C
B
F
z
v
C
u
B
C
y
w
C
x

- What is the complexity of DFS algorithm?

Thank you